Planning Completo — Cyberpedia Antiscam Tool

Tutte le richieste cliente + verifiche codice + skill

FASE A — Screen 1: Copy & CTA

A1 — Titolo schermata 1

Richiesta: "Fermati. Respira." in grassetto + "Prima **verifica**. Poi decidi." con "verifica" in grassetto turchese

Verifica codice Claude:

```
Leggi il componente della schermata iniziale e mostrami
esattamente il JSX del titolo. Verifica:
- "Fermati. Respira." ha font-bold o equivalente
- "verifica" ha colore turchese (es. text-cyan-400) e font-bold
- "Prima" e "Poi decidi." sono peso normale
```

Skills: `ui-ux-pro-max` `frontend-design` `interaction-design` `react-patterns` `typescript-advanced-types` `verification-before-completion`

A2 — Sottotitolo schermata 1

Richiesta: "Ti aiutiamo a vedere i segnali e fare la prossima mossa giusta, prima che il danno diventi reale."

Verifica codice Claude:

```
Mostrami il JSX del sottotitolo della schermata iniziale.
Verifica che il testo sia esattamente quello richiesto,
senza variazioni.
```

Skills: `frontend-design` `verification-before-completion`

A3 — CTA dinamica

Richiesta:

- Senza salvataggio → "Inizia il check (60 sec)"
- Con salvataggio → "Apri la tua difesa"

Verifica codice Claude:

```
Mostrami la logica che determina il testo della CTA
principale in schermata 1. Verifica:
- Legge correttamente lo stato di salvataggio da localStorage
- Mostra "Inizia il check (60 sec)" se non salvato
- Mostra "Apri la tua difesa" se salvato
- Il cambio è reattivo (non richiede reload)
```

Skills: `react-patterns` `react-state-management` `typescript-advanced-types` `verification-before-completion`

A4 — Lucchetto + privacy copy + link Dettagli

Richiesta: Sotto la CTA: icona lucchetto + "Privacy: tutto resta sul tuo telefono." + link "(Dettagli)" — presente in ENTRAMBE le fasi (prima e dopo salvataggio)

Verifica codice Claude:

```
Mostrami il JSX dell'area sotto la CTA in schermata 1.
Verifica:
- Icona lucchetto presente
- Testo "Privacy: tutto resta sul tuo telefono." presente
- Link "(Dettagli)" presente e cliccabile
- Tutto visibile sia prima che dopo il salvataggio
```

Skills: `ui-ux-pro-max` `interaction-design` `frontend-design` `react-patterns` `verification-before-completion`

FASE B — Screen 2: Layout generale

B1 — Rimozione "ultimo salvataggio"

Richiesta: Rimuovere l'elemento "ultimo salvataggio" in alto a destra

Verifica codice Claude:

```
Cerca in tutti i file JSX/TSX la stringa "ultimo salvataggio"
o "lastSaved" o "last_saved".
```

```
Verifica che non venga renderizzato nessun elemento
con questo testo in alto a destra della schermata 2.
```

Skills: `react-patterns` `verification-before-completion` `systematic-debugging`

B2 — Tab Base / Tab Scenario (checklist)

Richiesta : Separare le azioni con Tab A "Base (sempre)" e Tab B "Scenario (mirato)"

Verifica codice Claude:

```
Mostrami il componente che gestisce la checklist delle azioni.
Verifica:
- Esistono due tab distinti: "Base" e "Scenario"
- Tab Base: contiene azioni generiche sempre visibili
- Tab Scenario: contiene azioni mirate al tipo di truffa selezionato
- La navigazione tra tab funziona senza perdere lo stato
- La tab attiva è visivamente distinguibile
```

Skills: `ui-ux-pro-max` `react-patterns` `react-state-management` `interaction-design` `tailwind-css-patterns` `shadcn-ui` `typescript-advanced-types` `verification-before-completion`

B3 — Azioni come popup in prima pagina

Richiesta : Le azioni devono apparire come popup in prima pagina invece che a scorrimento

Verifica codice Claude:

```
Mostrami come vengono mostrate le azioni/checklist.
Verifica:
- Le azioni appaiono in un modal o bottom sheet overlay
- Non richiedono scroll verticale per essere raggiunte
- Il popup si apre dalla prima pagina visibile
- Il popup ha un sistema di chiusura (X o tap fuori)
```

Skills: `ui-ux-pro-max` `interaction-design` `react-patterns` `react-state-management` `tailwind-css-patterns` `verification-before-completion`

FASE C — Step 1: Contatti rapidi (redesign completo)

C1 — Campo "Nome banca" + label corretta

Richiesta : Campo "Nome banca" separato + label numero cambiata in "Numero assistenza banca/carta" + testo esplicativo sopra

Verifica codice Claude:

```
Mostrami il JSX del form Step 1.
Verifica:
- Esiste campo input con placeholder/label "Nome banca"
- Il campo numero è etichettato "Numero assistenza banca/carta"
- C'è un testo descrittivo sopra il campo numero che spiega
  perché inserirlo (es. "Inserisci il numero della tua banca
  per trovarlo subito disponibile in caso di emergenza")
```

Skills: `ui-ux-pro-max` `frontend-design` `interaction-design` `react-patterns` `typescript-advanced-types` `verification-before-completion`

C2 — Bandierina + prefisso separato dal numero

Richiesta: Input numero internazionale: bandierina + prefisso (es. +39) separato dal numero

Verifica codice Claude:

```
Mostrami il componente input telefono in Step 1.
Verifica:
- Selettore bandierina/prefisso separato dal campo numero
- Default: Italia (+39)
- Il prefisso non è incluso nel valore salvato in localStorage
  come parte del numero grezzo (o è gestito separatamente)
- Layout compatto (non occupa troppa altezza)
```

Skills: `ui-ux-pro-max` `react-patterns` `react-state-management` `typescript-advanced-types` `tailwind-css-patterns` `interaction-design` `verification-before-completion`

C3 — Icona edit/salva inline (no bottone finale)

Richiesta: Sostituire il bottone "Salva" finale con un'icona edit/salva accanto al campo

Verifica codice Claude:

```
Mostrami il JSX del campo numero banca.
Verifica:
- Non esiste un bottone standalone "Salva" per questo campo
- Esiste un'icona (edit o check) accanto al campo
- In modalità view: icona matita → entra in edit
- In modalità edit: icona check/save → salva e torna in view
- Lo stato saved/editing è gestito localmente per ogni campo
```

Skills: `ui-ux-pro-max` `interaction-design` `react-patterns` `react-state-management` `typescript-advanced-types` `verification-before-completion`

C4 — CTA verde "[allerta la banca]" cliccabile

Richiesta: Dopo il salvataggio del numero banca, appare CTA verde "[allerta la banca]" che apre chiamata in schermata esterna

Verifica codice Claude:

```
Mostrami il JSX dell'area numero banca dopo il salvataggio.
Verifica:
- CTA "[allerta la banca]" appare SOLO se il numero è salvato
- Il colore è verde (es. bg-green-500 o equivalente)
- Il click apre tel: link con il numero salvato
  (es. <a href="tel:+39...">)
- Su desktop mostra un prompt di conferma, su mobile apre
  direttamente il dialer
```

Skills: `ui-ux-pro-max` `interaction-design` `react-patterns` `typescript-advanced-types` `tailwind-css-patterns` `verification-before-completion`

C5 — Numero Polizia Postale / link commissariato

Richiesta: Aggiungere numero Polizia Postale (800 POSTAL o link commissariato online)

Verifica codice Claude:

```
Cerca in Step 1 il riferimento alla Polizia Postale.
Verifica:
- Presente numero Polizia Postale (800 288 883 o equivalente)
  o link a commissariatodips.it
- Visibile senza dover salvare nulla (sempre presente)
- Se è un link, apre in nuova tab
```

Skills: `frontend-design` `interaction-design` `react-patterns` `verification-before-completion`

C6 — Persistenza contatti al riavvio (BUG CRITICO)

Richiesta: I contatti salvati devono essere ripristinati al riavvio dell'app

Verifica codice Claude:

```
Questo è un bug critico. Analizza il sistema di storage
dei contatti in Step 1.
Verifica:
1. Mostrami dove viene fatto il write su localStorage
   (o altro storage)
2. Mostrami dove viene fatto il read/hydration all'avvio
3. Verifica che la hydration avvenga PRIMA del primo render
4. Testa manualmente: salva un numero → ricarica la pagina
   → il numero deve essere ancora presente
```

```
5. Mostrami l'output di localStorage.getItem('[chiave]')
   dalla console dopo il salvataggio

Usa /systematic-debugging per analizzare il flusso completo
read → state → render.
```

Skills: `systematic-debugging` `react-patterns` `react-state-management` `typescript-advanced-types` `debugging-strategies` `verification-before-completion`

FASE D — Step 2: Piano rapido

D1 — Solo 5 azioni salvavita

Richiesta: Step 2 deve contenere SOLO queste 5 azioni, nient'altro:

1. Non inviare denaro/codici/documenti
2. Metti pausa di 24 ore (o "prendi tempo")
3. Verifica con un terzo (contatto fiducia)
4. Cambia password/2FA se hai condiviso dati
5. Contatta banca se c'è transazione o dati bancari

Verifica codice Claude:

```
Mostrami il contenuto completo di Step 2.
Verifica:
- Contiene ESATTAMENTE 5 azioni, non di più
- I testi corrispondono esattamente alle 5 richieste
- Non ci sono altri elementi/azioni extra
- Ogni azione è spuntabile come checklist
```

Skills: `frontend-design` `react-patterns` `typescript-advanced-types` `verification-before-completion`

D2 — Checklist differenziata prevenzione vs riparazione

Richiesta: Distinguere azioni di prevenzione da azioni di riparazione (truffa già avvenuta)

Verifica codice Claude:

```
Mostrami come sono categorizzate le azioni nelle tab
Base e Scenario.
Verifica:
- Tab Base include sia azioni prevenzione che riparazione
  chiaramente etichettate
- Le azioni di riparazione hanno indicazione visiva
  "se è già successo" o equivalente
```

```
- Il tipo di azioni mostrate cambia in base alla risposta
  "Hai subito una truffa? Sì/No"
```

Skills: `ui-ux-pro-max` `interaction-design` `react-state-management` `typescript-advanced-types` `verification-before-completion`

FASE E — Step 3: Scenario preferito

E1 — Copy ridisegnato e selezione singola

Richiesta : "Se vuoi, scegli lo scenario più probabile: così ti prepariamo la checklist mirata." — selezione singola, poi CTA "Prova una simulazione (45 sec)" e "Salva"

Verifica codice Claude:

```
Mostrami il JSX completo di Step 3.
Verifica:
- Il copy introduttivo è esattamente quello richiesto
- La selezione è singola (radio, non checkbox)
- Esistono due CTA: "Prova una simulazione (45 sec)" e "Salva"
- Non c'è una checklist lunga in questa schermata
```

Skills: `ui-ux-pro-max` `frontend-design` `react-patterns` `interaction-design` `verification-before-completion`

E2 — "Truffa sentimentale" pre-selezionata + altre grigie/beta

Richiesta : Romance scam pre-selezionata di default, le altre hanno badge "Prossimamente" o "Beta" e sono visivamente disabilitate/grigie

Verifica codice Claude:

```
Mostrami le card dei tipi di attacco in Step 3.
Verifica:
- "Truffa sentimentale" (romance-scam) è pre-selezionata
  al caricamento
- Le altre card hanno opacity ridotta o stile grigio
- Le altre card hanno badge "Prossimamente" o "Beta"
- Le altre card sono cliccabili o disabilitate?
  (deve essere specificato)
```

Skills: `ui-ux-pro-max` `tailwind-css-patterns` `react-patterns` `interaction-design` `verification-before-completion`

E3 — Rinomina "Ingegneria sociale"

Richiesta: "Ingegneria sociale" → "Manipolazione psicologica" o "Pressione e urgenza"

Verifica codice Claude:

```
Cerca in tutti i file la stringa "Ingegneria sociale"
o "social-engineering" o equivalente.
Verifica che nelle card utente visibili sia usato
"Manipolazione psicologica" o "Pressione e urgenza",
non il termine tecnico.
```

Skills: `frontend-design` `verification-before-completion`

E4 — Salvataggio statistiche tipo truffa selezionato

Richiesta: Quando l'utente seleziona un tipo di truffa, la scelta deve essere salvata per le statistiche

Verifica codice Claude:

```
Mostrami dove viene gestita la selezione del tipo di truffa.
Verifica:
- Al momento della selezione viene salvato un record
  (localStorage, Supabase o equivalente) con:
  { attackType, timestamp, sessionId o userId anonimo }
- Il salvataggio avviene anche se l'utente non completa
  il wizard
- Mostrami il codice della funzione di salvataggio
```

Skills: `react-state-management` `typescript-advanced-types` `nodejs-backend-patterns` `verification-before-completion`

FASE F — Modale riapertura app

F1 — Modale "Hai subito una truffa? Sì/No"

Richiesta: Alla riapertura, dopo la home, chiedere "Hai subito una truffa?" con biforcazione Sì/No

Verifica codice Claude:

```
Mostrami il componente che gestisce la riapertura dell'app
(quando l'utente ha già salvato dati).
Verifica:
- Appare un modale/schermata con la domanda
  "Hai subito una truffa?"
- Due opzioni: "Sì" e "No"
```

```
- "No" → mostra azioni di prevenzione + selezione tipo attacco
- "Sì" → mostra azioni di intervento + selezione tipo attacco
- La biforcazione è persistente per la sessione corrente
```

Skills: `ui-ux-pro-max` `react-patterns` `react-state-management` `interaction-design` `typescript-advanced-types` `verification-before-completion`

FASE G — Modalità al bisogno

G1 — Schermata "Modalità al bisogno" con Tab Base + Tab Mirato

Richiesta: Schermata con Tab Base + Tab Mirato + CTA "Crea report" + "Chiama banca" se configurato

Verifica codice Claude:

```
Mostrami il componente della "Modalità al bisogno".
Verifica:
- Esistono Tab "Base" e Tab "Mirato"
- CTA "Crea report" presente e funzionante
- CTA "Chiama banca" appare SOLO se il numero banca
  è stato salvato in Step 1
- "Chiama banca" usa il numero salvato (tel: link)
```

Skills: `ui-ux-pro-max` `react-patterns` `react-state-management` `interaction-design` `tailwind-css-patterns` `typescript-advanced-types` `verification-before-completion`

FASE H — Micro-frizione intelligente

H1 — Popup contestuale su azioni gravi

Richiesta: Quando l'utente spunta un'azione grave (es. "ho condiviso dati bancari", "ho inviato denaro"), appare un box micro-azione con CTA "Chiama banca" o "Copia numero"

Verifica codice Claude:

```
Mostrami il sistema di gestione delle spunte nella checklist.
Verifica:
- Esiste una lista di azioni "gravi" definita nel codice
- Quando una di queste viene spuntata, appare un box/popup
  con testo contestuale
- Il box ha CTA "Chiama banca" (se numero salvato)
  o "Copia numero" (se non salvato)
- Il popup non blocca il resto dell'interfaccia
  (non è un modal fullscreen)
- Il popup è dismissibile
```

Skills: `ui-ux-pro-max` `interaction-design` `react-patterns` `react-state-management` `typescript-advanced-types` `tailwind-css-patterns` `verification-before-completion`

FASE I — Simulazioni: redesign completo

I1 — Prima simulazione solo opzioni corrette (onboarding guidato)

Richiesta: Prima simulazione = solo opzioni corrette (tutte giuste, nessuna sbagliata). Dalla seconda in poi anche una sbagliata

Verifica codice Claude:

```
Mostrami come viene determinato se mostrare opzioni
solo corrette o anche sbagliate.
Verifica:
- Esiste un contatore "simulationCount" in localStorage
  o equivalente
- Se simulationCount === 0 (prima simulazione):
  le opzioni mostrate sono tutte correct:true
- Se simulationCount > 0: viene inclusa 1 opzione
  correct:false
- Il contatore viene incrementato al completamento
  della simulazione
- Dopo la prima simulazione appare il messaggio
  "Prova ora un'opzione più realistica" o equivalente
```

Skills: `react-patterns` `react-state-management` `typescript-advanced-types` `interaction-design` `verification-before-completion`

I2 — Due risposte giuste con competenze diverse

Richiesta: Le due risposte corrette devono allenare competenze diverse (es. "limite/NO" vs "verifica/prova di realtà"), non essere due varianti della stessa cosa

Verifica codice Claude:

```
Leggi i file delle simulazioni statiche esistenti
e il prompt Gemini in N8N.
Verifica:
- Ogni SimChoice ha 2 opzioni correct:true
  con funzioni diverse (una = limite, una = verifica)
- Le label non sono sinonimi (es. non
  "Non lo faccio" e "Non voglio farlo")
- Nel prompt N8N è specificato esplicitamente che le due
  risposte giuste devono allenare competenze diverse
```

Skills: `prompt-engineering` `prompt-engineering-patterns` `typescript-advanced-types` `verification-before-completion`

I3 — Risposta sbagliata sempre in terza posizione e meno prominente

Richiesta: L'opzione `correct: false` deve essere sempre visualizzata in terza posizione e con stile visivamente meno invitante

Verifica codice Claude:

```
Mostrami il componente che renderizza le opzioni di scelta.
Verifica:
- Le opzioni vengono ordinate prima del render:
  le due corrette in posizione 1 e 2, la sbagliata in 3
- L'opzione in posizione 3 ha uno stile diverso
  (es. opacity più bassa, bordo meno prominente,
  assenza di hover colorato)
- L'ordinamento non è casuale ma deterministico (sbagliata = ultima)
- Verifica che nel codice non ci sia un .sort() o shuffle
  che potrebbe rimescolare le posizioni
```

Skills: `react-patterns` `ui-ux-pro-max` `tailwind-css-patterns` `typescript-advanced-types` `verification-before-completion`

I4 — Feedback corto e prescrittivo

Richiesta:

- Se sbaglia → "Stop. Questa è la trappola." + 1 riga rischio + 1 riga mossa giusta + CTA "Riprova"
- Se giusta → "Corretto." + 1 riga regola + 1 riga rinforzo + CTA "Avanti"
- Il testo lungo va in "Approfondisci" facoltativo

Verifica codice Claude:

```
Mostrami il componente che renderizza il feedback
(SimFeedback / Attenzione / Corretto).
Verifica:
- Feedback sbagliato: titolo "Stop. Questa è la trappola."
- Feedback sbagliato: max 2 righe di testo
- Feedback sbagliato: CTA "Riprova" presente
- Feedback corretto: titolo "Corretto."
- Feedback corretto: max 2 righe di testo
- Feedback corretto: CTA "Avanti" presente
- Se esiste testo lungo educativo, è nascosto sotto
  "Approfondisci" collassabile
```

Skills: `ui-ux-pro-max` `frontend-design` `interaction-design` `react-patterns` `typescript-advanced-types` `verification-before-completion`

I5 — Dopo errore: una sola frase di manipolazione poi scelta immediata

Richiesta: Dopo risposta sbagliata, mostrare UNA sola frase del truffatore (breve, manipolativa) poi subito le scelte

Verifica codice Claude:

```
Mostrami il flusso post-errore nel ChatSimulator.
Verifica:
- Dopo risposta sbagliata appare 1 sola frase (retryMessage)
- La frase ha max 1-2 righe
- Dopo la frase appaiono immediatamente le scelte
  (nessun delay eccessivo, nessun testo aggiuntivo)
- Non appaiono più frasi in sequenza dopo l'errore
```

Skills: `react-state-management` `interaction-design` `typescript-advanced-types` `verification-before-completion`

I6 — 3 simulazioni statiche aggiornate con il copy del cliente

Richiesta: Aggiornare (o creare) le 3 simulazioni con il copy esatto fornito dal cliente:

- Sim 1: "Carta clonata all'estero"
- Sim 2: "Dogana / pacco"
- Sim 3: "Ti mando i documenti"

Verifica codice Claude:

```
Leggi i file delle simulazioni statiche esistenti.
Verifica che esistano queste 3 simulazioni con:

SIM 1 — romance-scam / carta clonata:
- Messaggio apertura: "Ciao tesoro, penso sempre a te..."
- Scelta 1 opzione A: "Certo, te li mando subito..."  (pos 3, sbagliata)
- Scelta 1 opzione B: "No. Non invio denaro..." (corretta: limite)
- Scelta 1 opzione C: "Possiamo fare una videochiamata..." (corretta: verifica)
- Feedback sbagliata: "Stop. Questa è la trappola. / Urgenza + richiesta = rischio alto.
- Proseguimento: "Mi fai sentire come se non ti fidassi..."
- Scelta 2 con A sbagliata (pos 3), B e C corrette

SIM 2 — dogana/pacco:
- Struttura analoga con copy esatto del cliente
- Gift card come metodo = sbagliata in pos 3

SIM 3 — richiesta dati/documenti:
- Struttura analoga con copy esatto del cliente
- Inviare documenti in chat = sbagliata in pos 3

Mostrami i file completi di tutte e 3 le simulazioni.
```

Skills: `typescript-advanced-types` `frontend-design` `prompt-engineering` `verification-before-completion`

FASE J — Refinement grafico

J1 — Luminosità testo descrittivo

Richiesta: Aumentare leggermente la luminosità del testo descrittivo (sottotitoli, description, hint text)

Verifica codice Claude:

```
Mostrami le classi Tailwind usate per il testo descrittivo
in tutta l'app (cerca text-gray-*, text-slate-*, text-muted-*).
Verifica:
- I testi descrittivi usano almeno text-gray-300 o equivalente
  (non text-gray-500 o più scuro su sfondi dark)
- Mostrami un before/after delle classi modificate
```

Skills: `tailwind-css-patterns` `tailwind-design-system` `ui-ux-pro-max` `visual-design-foundations` `verification-before-completion`

J2 — Riduzione bordi concentrici

Richiesta: Le card hanno troppi bordi concentrici — ridurre a 1 bordo + shadow soft

Verifica codice Claude:

```
Mostrami le classi delle card principali dell'app.
Verifica:
- Nessuna card ha più di 1 bordo (ring + border insieme
  sulla stessa card = da rimuovere)
- Aggiunta shadow soft (es. shadow-md o shadow-lg)
- Rimuovi ring se già c'è border, o viceversa
```

Skills: `tailwind-css-patterns` `tailwind-design-system` `visual-design-foundations` `ui-ux-pro-max` `verification-before-completion`

J3 — Gerarchia bottoni Indietro / Salva / Avanti

Richiesta:

- "Avanti" → primario (pieno, colore principale)
- "Salva" → secondario (bordo, no fill)
- "Indietro" → terziario (link/ghost, più piccolo)

Verifica codice Claude:

```
Mostrami il componente dei bottoni di navigazione
bottom in Step 1, 2, 3.
Verifica:
- "Avanti": bg-primary o equivalente, peso visivo massimo
- "Salva": variant outline o ghost con colore accent,
  non compete con Avanti
- "Indietro": testo puro o variant ghost, dimensione
  ridotta (text-sm) o senza background
- Su mobile tutti e 3 sono raggiungibili senza scroll
```

Skills: `ui-ux-pro-max` `tailwind-css-patterns` `interaction-design` `visual-design-foundations` `shadcn-ui` `verification-before-completion`

FASE K — Deploy finale

K1 — Build check completo

```
Prima del deploy finale esegui in sequenza:
1. npm run type-check → EXIT:0
2. npm run lint → 0 errori, 0 warning
3. npm run build → 0 errori TypeScript, 0 warning

Se uno step fallisce: fermati, mostrami l'errore completo,
non procedere al deploy.
Usa /verification-before-completion.
```

Skills: `verification-before-completion` `systematic-debugging` `debugging-strategies`

K2 — Deploy Cloudflare Pages

```
Esegui:
git add .
git status (mostrami i file modificati)
git commit -m "feat: complete UI redesign + AI simulations +
  emergency contacts fix + onboarding guided simulations"
git push → attendi auto-deploy Cloudflare Pages

Usa /finishing-a-development-branch +
/deployment-pipeline-design.
```

Skills: `finishing-a-development-branch` `deployment-pipeline-design` `git-advanced-workflows` `verification-before-completion`

Tabella riepilogativa priorità

Fase	Richiesta	Priorità	Bloccante deploy?
C6	Persistenza contatti al riavvio	Alta	Sì
F1	Modale Sì/No alla riapertura	Alta	Sì
H1	Micro-frizione intelligente	Alta	No
I4	Feedback corto prescrittivo	Alta	No
I6	3 simulazioni con copy cliente	Alta	No
A1–A4	Copy schermata 1 + privacy	Media	No
B1	Rimozione ultimo salvataggio	Media	No
B2	Tab Base / Scenario	Media	No
C1–C5	Redesign contatti rapidi	Media	No
D1–D2	Step 2 solo 5 azioni	Media	No
E1–E4	Step 3 redesign + stats	Media	No
G1	Modalità al bisogno	Media	No
I1–I3	Onboarding guidato + posizioni	Media	No
I5	Una frase post-errore	Media	No
J1–J3	Refinement grafico	Bassa	No
B3	Popup azioni prima pagina	Bassa	No